

CS 584 Natural Language Processing

Lecture 10: Syntax and Discourse

Yue Ning
Department of Computer Science
Stevens Institute of Technology

Today's Lecture and Learning Objectives

Lecture Plan:

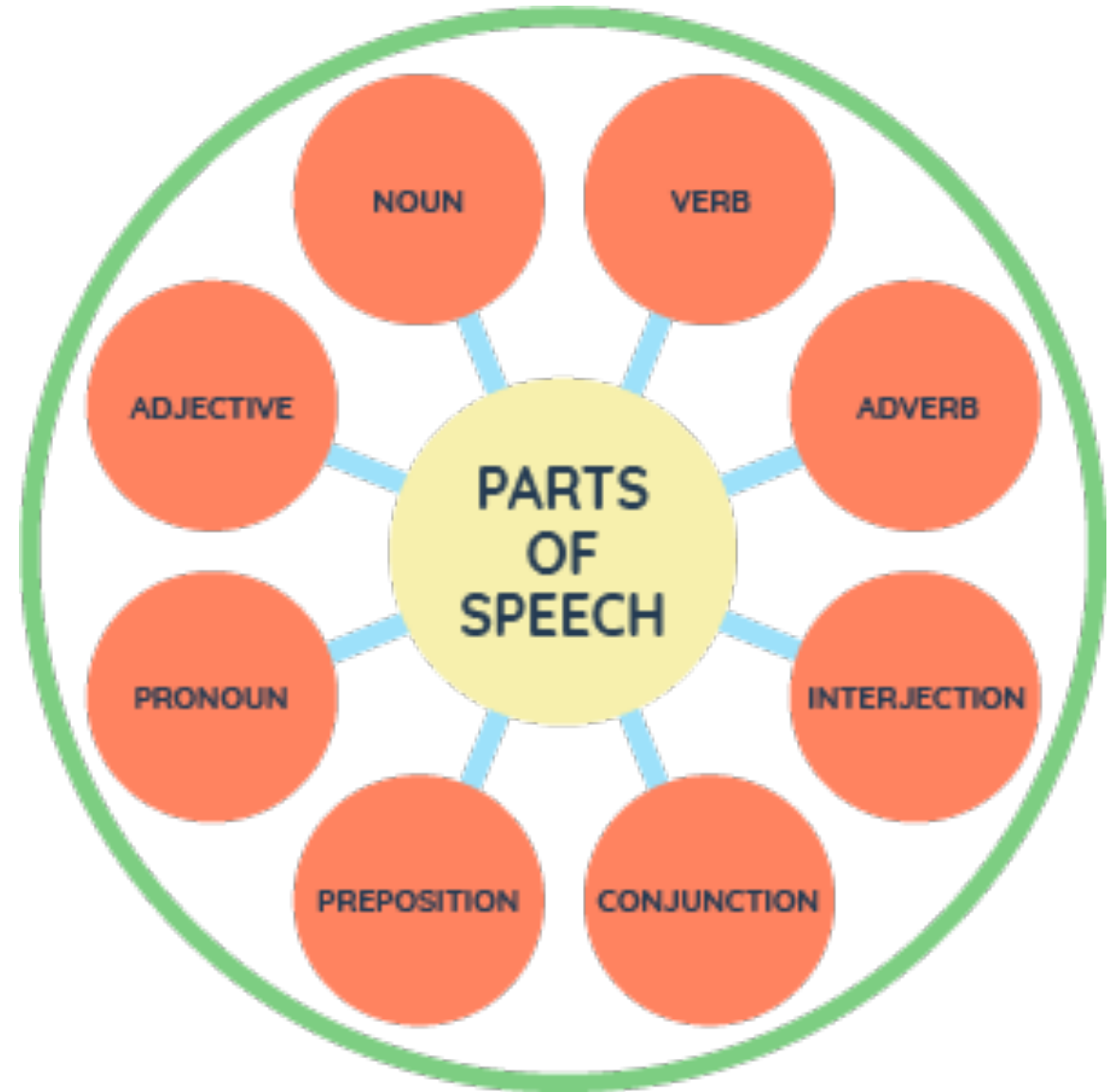
- Syntax
 - Part-of-speech tagging
 - Constituency and dependency parsing
- Discourse
 - Rhetorical structure theory

Learning objectives:

Be able to explain dependency parsing



Part-of-speech



Part-of-speech

Categories of words based on their grammatical properties.

Following are some popular POS categories from [UD version 2](#):

	Tag	POS Category	Example Words
Open-class	NOUN	Noun	book, tree, car
	VERB	Verb	run, eat, is
	ADJ	Adjective	big, red, happy
	ADV	Adverb	quickly, very
Closed-class	PRON	Pronoun	I, you, he, she
	NUM	Numeral	one, two, III, IV
	CCONJ	Coordinating conjunction	and, or, but
	DET	Determiner	the, a, an

Part-of-speech tagging

POS tagging assigns a POS tag for each word in the sentence.

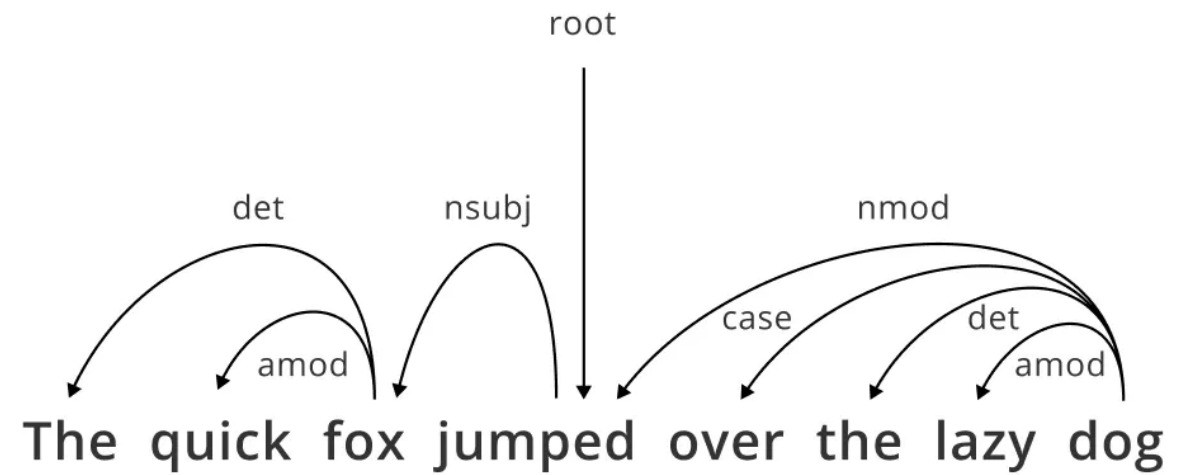
Example:

Input: “The quick brown fox jumps over the lazy dog.”

Output: DET ADJ ADJ NOUN VERB ADP DET ADJ NOUN

- Each word could have multiple POS tags.
- NLP models must consider the context to assign the correct POS tags.

Constituency and Dependency Parsing



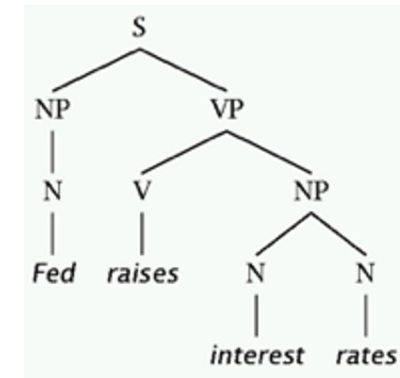
The need to sentence structures

- ❑ We need to understand sentence structure in order to be able to interpret language correctly
- ❑ Humans communicate complex ideas by **composing words together** into bigger units to convey complex meanings
 - ❑ *“Inside the carriage, which is built on several levels, he sits in velveteen darkness, with nothing to smoke, feeling metal nearer and farther rub and connect, steam escaping in puffs, a vibration in the carriage frame, a poising, an uneasiness, all the others pressed in around, feeble ones, second sheep, all out of luck and time: drunks, old veterans still in shock from...” - Gravity’s Rainbow*
- ❑ We need to know what is connected to what

Two views of linguistic structure:

1. Constituency (phrase structure)

- ❑ **Idea:** Sentences are broken down into hierarchical constituents (or "phrases")
- ❑ Phrase structure organizes words into nested constituents.
- ❑ How do we know what is a constituent? (Not that linguists don't argue about some cases.)
- ❑ Distribution: a constituent behaves as a unit that can appear in different places:
 - ❑ John talked [to the children] [about drugs].
 - ❑ John talked [about drugs] [to the children].
 - ❑ *John talked drugs to the children about
- ❑ Substitution/expansion/pro-forms:
 - ❑ I sat [on the box/right on top of the box/there].
- ❑ Coordination, regular internal structure, no intrusion, semantics, ...



Two views of linguistic structure:

1. Constituency (phrase structure)

- ❑ Phrase structure organizes words into nested constituents.
 - ❑ can represent the grammar with Context-Free Grammars (CFG) rules
- ❑ **Starting units:** words are given a category (part of speech = POS)
the, cuddly, cat, by, the, door
Det Adj N P Det N
- ❑ **Words combine into phrases** with categories
the cuddly cat, by the door
NP -> Det Adj N PP->P NP
- ❑ **Phrases can combine into bigger phrases** recursively
the cuddly cat by the door
NP -> NP PP

Headed phrase structure

- ❑ VP -> ... VB* ...
- ❑ NP -> NN* ...
- ❑ ADJP (adjective phrase)-> ...JJ*(adjective)...
- ❑ ADVP (adverb phrase)-> ...RB* (adverb)...

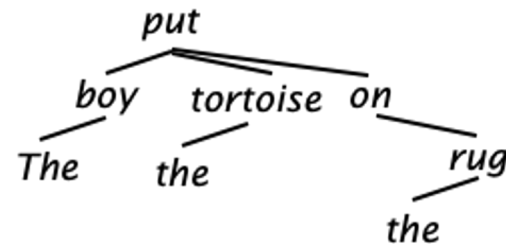
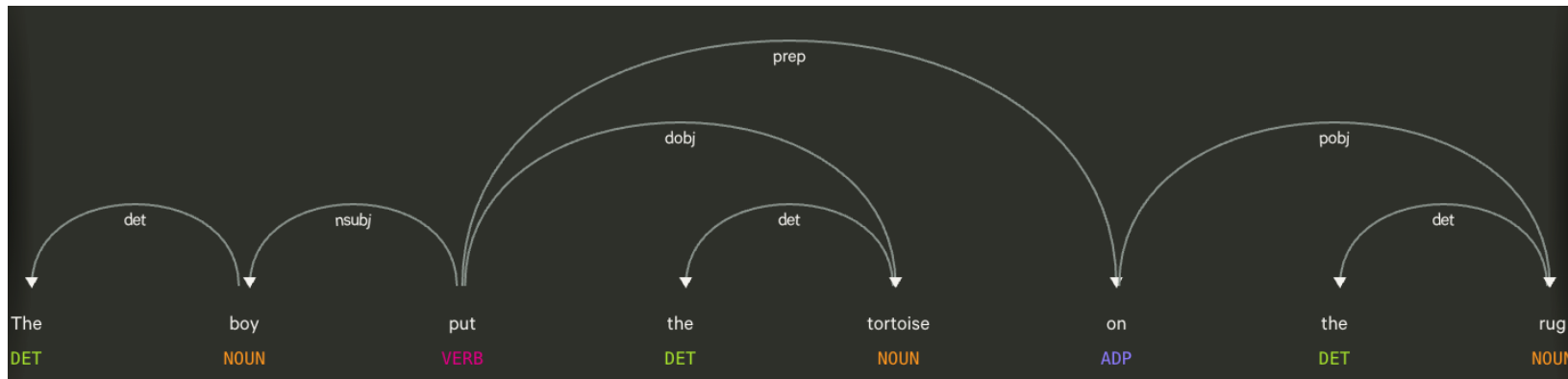
- ❑ SBAR(Q) (subordinating conjunction) ->
 - ❑ S|SINV (Inverted declarative sentence)|SQ (Inverted yes/no question) ->
 - ❑ ...NP VP ...

- ❑ Plus minor phrase types:
 - ❑ QP (quantifier phrase in NP), CONJP(multi word constructions: as well as), INTJ(interjections). etc

Two views of linguistic structure:

2. Dependency structure

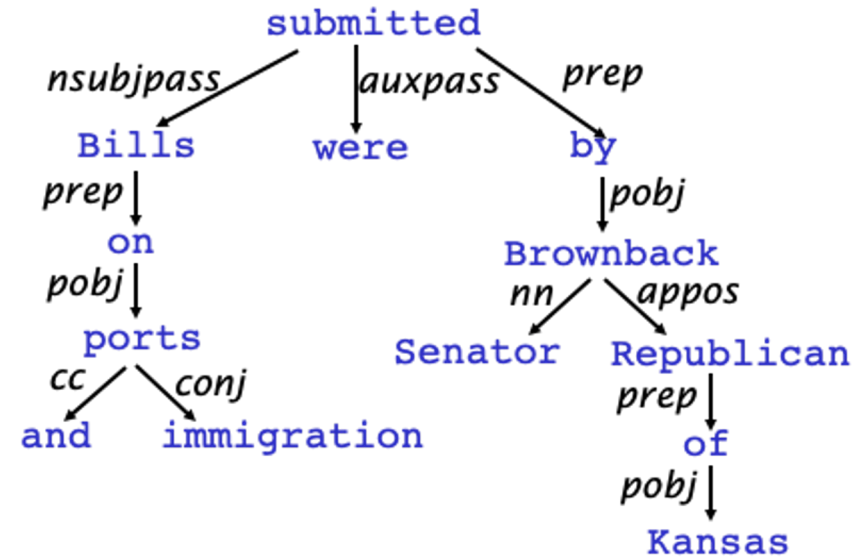
- ❑ **Idea:** Sentence structure is represented as a set of directed, grammatical relationships between words.
- ❑ Dependency structure shows which words depend on (modify or are arguments of) which other words.



Dependency grammar and structure

- Dependency syntax postulates that syntactic structure consists of lexical items linked by binary asymmetric relations (“arrows”) called **dependencies**

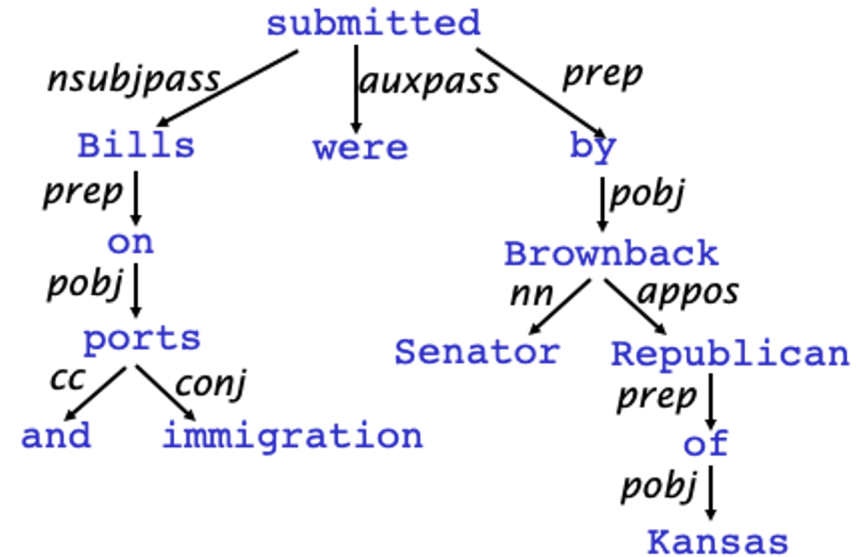
- The arrows are commonly **typed** with the name of grammatical relations (subject, prepositional object, apposition, etc.)



Dependency grammar and structure

- Dependency syntax postulates that syntactic structure consists of lexical items linked by binary asymmetric relations (“arrows”) called **dependencies**

- The arrow connects a **head** (governor, superior, regent) with a **dependent** (modifier, inferior, subordinate)
- Usually, dependencies form a tree (connected, acyclic, single-head)



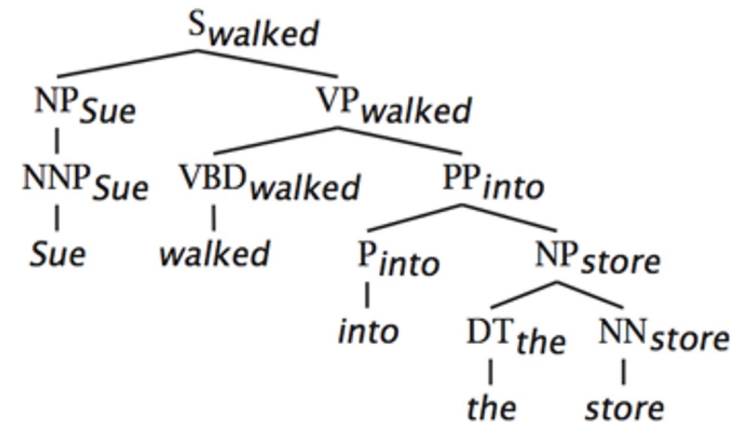
Constituency vs Dependency Parsing

Feature	Constituency Parsing	Dependency Parsing
Basic Unit	Phrases (constituents)	Words
Output	Tree showing phrase structure	Tree showing words' dependency relationships
Grammar	Context-Free Grammars (CFGs)	Dependency Grammars
	Has the notion of “sentence”	Has asymmetric relations and the notion of “head”

Extract dependency structure from phrase structure

- ❑ A dependency grammar has a notion of a head. Officially, CFGs don't.
- ❑ But modern linguistic theory and all modern statistical parsers (Charniak, Collins, Stanford, ...) do, via hand-written phrasal “head rules”:
 - ❑ The head of a Noun Phrase is a noun/number/adj/...
 - ❑ The head of a Verb Phrase is a verb/modal/....
- ❑ The head rules can be used to extract a dependency parse from a CFG parse

- The closure of dependencies give constituency from a dependency tree
- But the dependents of a word must be at the same level (i.e., “flat”)



Ambiguities in syntax structures



Why do we need sentence structure?

- ❑ We need to understand sentence structures in order to be able to **interpret language correctly.**
- ❑ Humans communicate complex ideas by **composing words together** into bigger units to convey complex meanings.
 - ❑ *“Inside the carriage, which is built on several levels, he sits in velveteen darkness, with nothing to smoke, feeling metal nearer and farther rub and connect, steam escaping in puffs, a vibration in the carriage frame, a poisoning, an uneasiness, all the others pressed in around, feeble ones, second sheep, all out of luck and time: drunks, old veterans still in shock from...” - Gravity’s Rainbow*
- ❑ We need to know what is connected to what

Prepositional phrase attachment ambiguity

- ❑ Arises in language when a prepositional phrase can be **attached to more than one part of a sentence**, resulting in different possible interpretations of the sentence's meaning.
- ❑ “San Jose cops kill man with knife”
- ❑ "The man saw the woman with the telescope."
- ❑ “Scientists count whales from space”



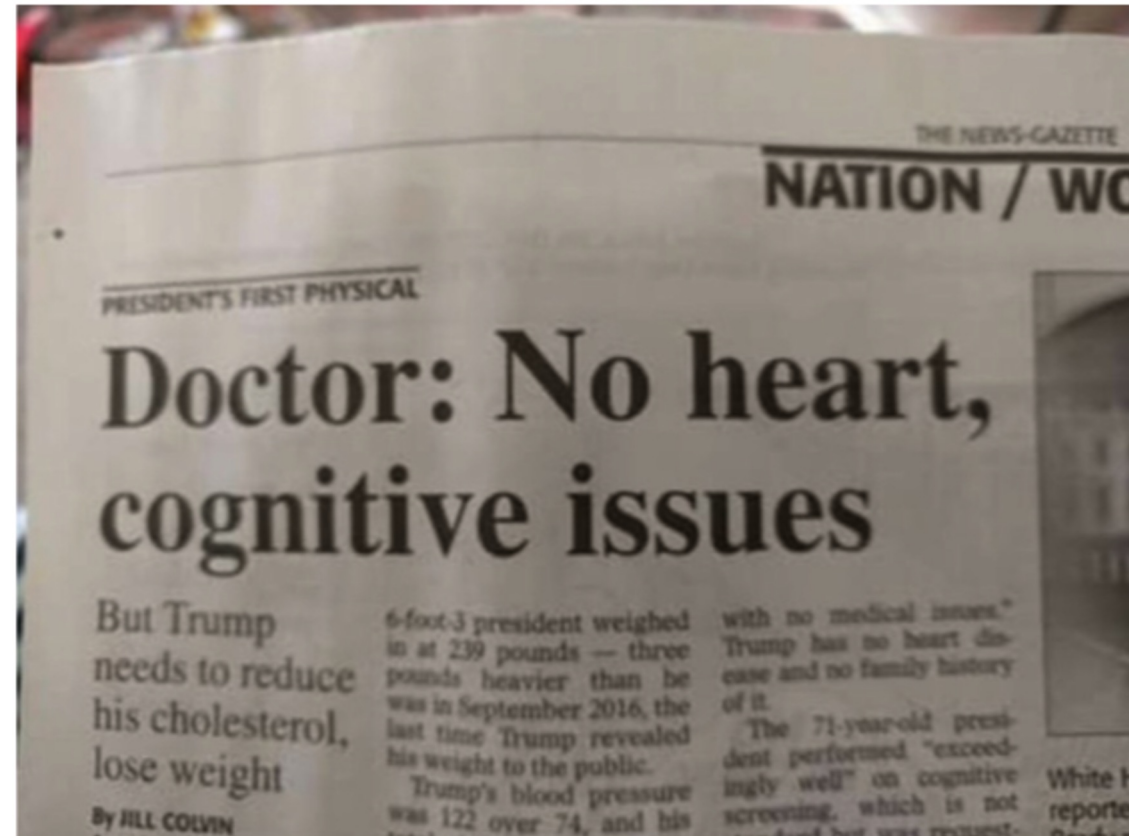
PP attachment ambiguities multiply

- ❑ A key parsing decision is how we ‘attach’ various constituents
 - ❑ PPs, adverbial or participial phrases, infinitives, coordinations
- *The board approved [its acquisition] [by Royal Trustco Ltd.] [of Toronto] [for \$27 a share] [at its monthly meeting]*

Coordination scope ambiguity

- A type of syntactic ambiguity that arises in sentences containing **coordinate structures**.
- Coordinate structures are sentences or phrases where two or more elements of similar syntactic type are joined together using coordinating conjunctions like "and," "or," or "but."
- The ambiguity occurs when it's not clear which elements are being coordinated with each other.
- An example: "Sue and John like apples and oranges."

Coordination scope ambiguity



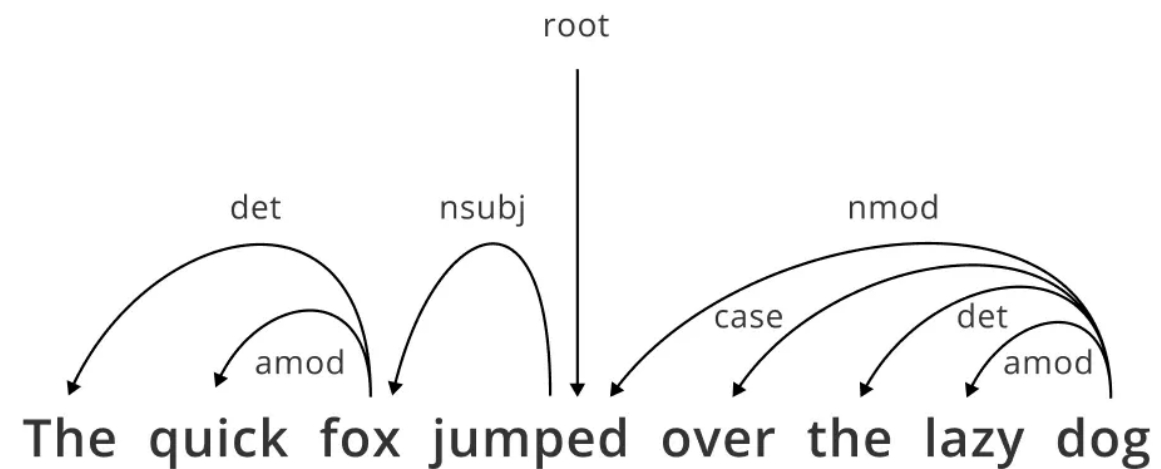
Coordination scope ambiguity

*Shuttle veteran and longtime NASA executive Fred Gregory
appointed to board*

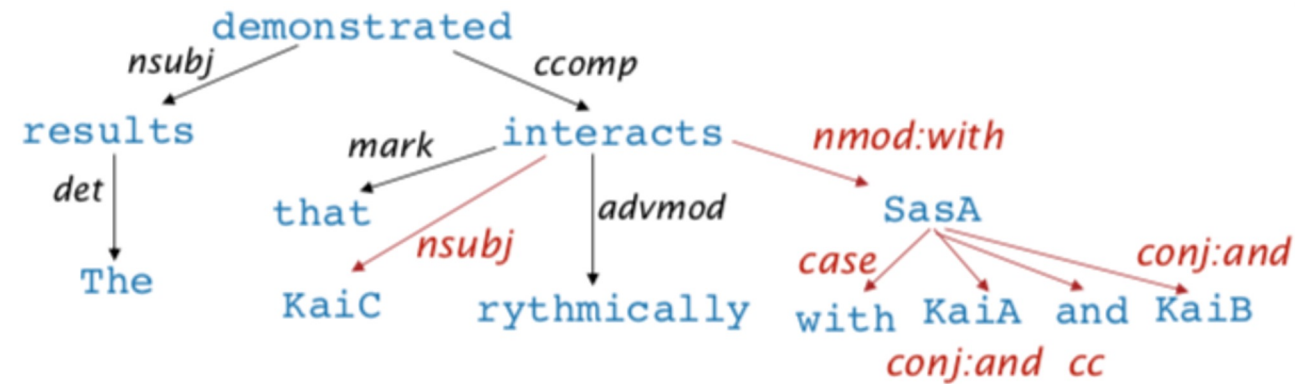
Verb phrase (VP) attachment ambiguity



Dependency Parsing



Dependency paths identify semantic relations e.g. for protein interaction



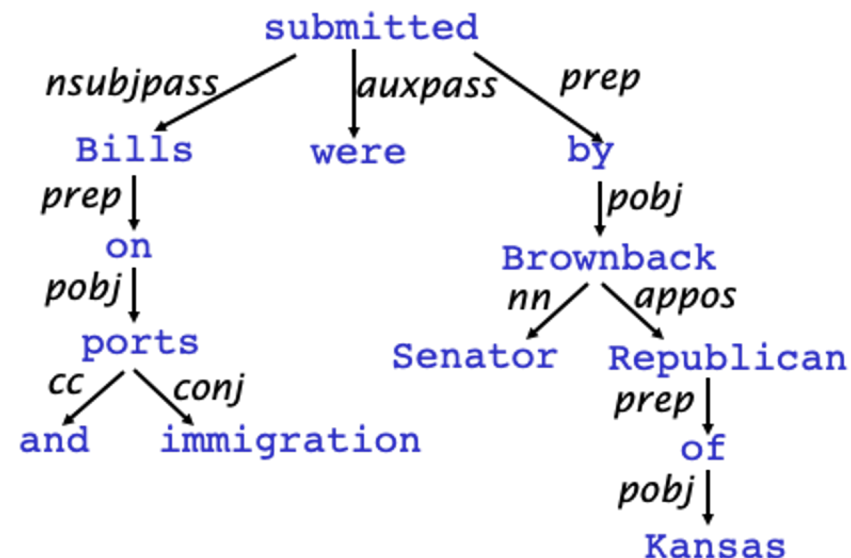
KaiC $\leftarrow$ nsubj interacts nmod:with $\rightarrow$ SasA

KaiC $\leftarrow$ nsubj interacts nmod:with $\rightarrow$ SasA conj:and $\rightarrow$ KaiA

KaiC $\leftarrow$ nsubj interacts prep_with $\rightarrow$ SasA conj:and $\rightarrow$ KaiB

Dependency grammar and structure

- ❑ Dependency syntax assumes that syntactic structure consists of **lexical items**, normally **binary asymmetric** relations (“arrows”) called **dependencies**
- ❑ The arrows are commonly **typed** with the name of grammatical relations ² (subject, prepositional object, apposition, etc.)
- ❑ The arrow connects a **head** (governor, superior, regent) with a **dependent** (modifier, inferior, subordinate)
- ❑ Usually, dependencies form a **tree** (connected, acyclic, single-head)



² https://downloads.cs.stanford.edu/nlp/software/dependencies_manual.pdf

Dependency Grammar and Dependency Structure



- The idea of dependency structure goes back a long way.
- Some people draw the arrows one way; some the other way!
 - In modern dependency work, people had them **point from head to dependent** – we follow that convention
 - The dependent modifies the head.
- We usually add a **fake ROOT** so every word is a dependent of precisely 1 other node

The rise of annotated data and Universal Dependencies treebanks

The Penn Treebank

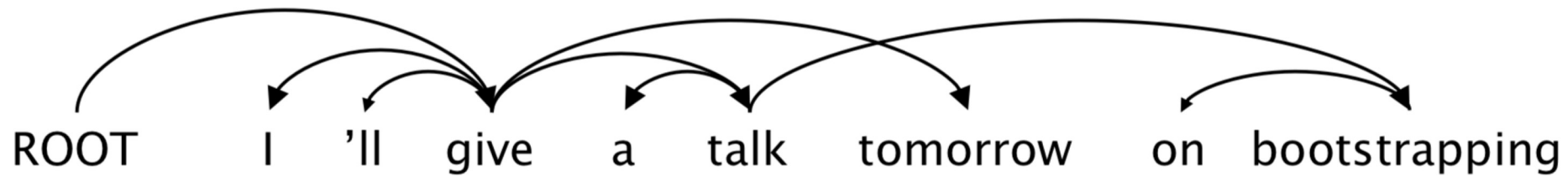
```
( (S
    (NP-SBJ (DT The) (NN move))
    (VP (VBD followed)
        (NP
            (NP (DT a) (NN round))
            (PP (IN of)
                (NP
                    (NP (JJ similar) (NNS increases))
                    (PP (IN by)
                        (NP (JJ other) (NNS lenders)))
                    (PP (IN against)
                        (NP (NNP Arizona) (JJ real) (NN estate) (NNS loans))))))
            (, ,)
            (S-ADV
                (NP-SBJ (-NONE- *))
                (VP (VBG reflecting)
                    (NP
                        (NP (DT a) (VBG continuing) (NN decline))
                        (PP-LOC (IN in)
                            (NP (DT that) (NN market))))))
            (. .)))
```

The rise of annotated data

- ❑ Starting off, building a treebank seems **a lot slower and less useful** than building a grammar
- ❑ But a treebank gives us many things
 - ❑ Reusability of the labor
 - ❑ Many parsers, POS taggers, etc. can be built on it
 - ❑ Valuable resource for linguistics
 - ❑ Broad coverage
 - ❑ Frequencies and distributional information
 - ❑ A way to evaluate systems

Dependency parsing

- The process to analyze the **grammatical structure** in a sentence and find out related words as well as the type of the **relationship** between them.
- Usually some **constraints**:
 - Only one word is a dependent of ROOT
 - Don't want cycles $A \rightarrow B, B \rightarrow A$
- This makes the dependencies a tree.
- Final issue is whether arrows can cross (**non-projective**) or not



Projectivity

- Definition of **a projective parse**: There are no crossing dependency arcs when the words are laid out in their linear order, with all arcs above the words.
- But dependency theory normally does allow non-projective structures to account for complex sentence structures.



Methods of dependency parsing

1. Dynamic programming

- a. Eisner ([1996](#)) gives a clever algorithm with complexity $O(n^3)$, by producing parse items with heads at the ends rather than in the middle

2. Graph algorithms

- a. Create a Minimum Spanning Tree for a sentence
- b. McDonald et al.'s ([2005](#)) MSTParser scores dependencies independently using an ML classifier (he uses Margin-infused relaxed algorithm MIRA, for online learning, but it can be something else)

3. Constraint Satisfaction

- a. Edges are eliminated that don't satisfy hard constraints. Karlsson ([1990](#)), etc.

4. “Transition-based parsing” or “deterministic dependency parsing”

- a. Greedy choice of attachments guided by good machine learning classifiers
MaltParser (Nivre et al. [2006](#)) has been proven highly effective.

Greedy transition-based parsing

[Nivre 2003]

- A simple form of greedy discriminative dependency parser
- The parser does **a sequence of bottom-up actions**
 - Roughly like “**shift**” or “**reduce**” in a shift-reduce parser, but the “reduce” actions are specialized to create dependencies with head on left or right
- The parser has:
 - **a stack σ** , written with top to the right: which starts with the ROOT symbol
 - **a buffer β** , written with top to the left: which starts with the input sentence
 - **a set of dependency arcs A** : which starts off empty
 - **a set of actions**

Basic transition-based dependency parser

- **Start** $\sigma = [\text{ROOT}]$, $\beta = w_1, \dots, w_n$, $A = \emptyset$
 1. Shift $\sigma, w_i \mid \beta, A \rightarrow \sigma \mid w_i, \beta, A$
 2. Left-Arc_r $\sigma \mid w_i \mid w_j, \beta, A \rightarrow \sigma \mid w_j, \beta, A \cup \{r(w_j, w_i)\}$
 3. Right-Arc_r $\sigma \mid w_i \mid w_j, \beta, A \rightarrow \sigma \mid w_i, \beta, A \cup \{r(w_i, w_j)\}$
- **Finish**: $\sigma = [\text{ROOT}]$, $\beta = \emptyset$

Arc-standard transition-based parser

Analysis of “I ate fish”

Start



Start: $\sigma = [\text{ROOT}]$, $\beta = w_1, \dots, w_n$, $A = \emptyset$

1. Shift $\sigma, w_i | \beta, A \rightarrow \sigma | w_i, \beta, A$
2. Left-Arc_r $\sigma | w_i | w_j, \beta, A \rightarrow \sigma | w_j, \beta, A \cup \{r(w_j, w_i)\}$
3. Right-Arc_r $\sigma | w_i | w_j, \beta, A \rightarrow \sigma | w_i, \beta, A \cup \{r(w_i, w_j)\}$

Finish: $\beta = \emptyset$

Arc-standard transition-based parser

Analysis of “I ate fish”

Left Arc



A +=
nsubj(ate → I)

Shift-Reduce: A Dependency Parser

Input: a sentence “I eat fish”

Output: a collection of “arcs” (dependency relations)

Step	Stack	Buffer	Action	Arcs
0	[ROOT]	[I, eat, fish]	SHIFT	$\emptyset$
1	[ROOT, I]	[eat, fish]	SHIFT	$\emptyset$
2	[ROOT, I, eat]	[fish]	LEFT-ARC(nsubj)	{(eat $\rightarrow$ I)}
3	[ROOT, eat]	[fish]	SHIFT	{(eat $\rightarrow$ I)}
4	[ROOT, eat, fish]	[]	RIGHT-ARC(obj)	{(eat $\rightarrow$ I), (eat $\rightarrow$ fish)}
5	[ROOT, eat]	[]	RIGHT-ARC(root)	{(eat $\rightarrow$ I), (eat $\rightarrow$ fish), (ROOT $\rightarrow$ eat)}

Arc-standard transition-based parser

Now, it is your turn!

Analysis of “**The dog saved me**”

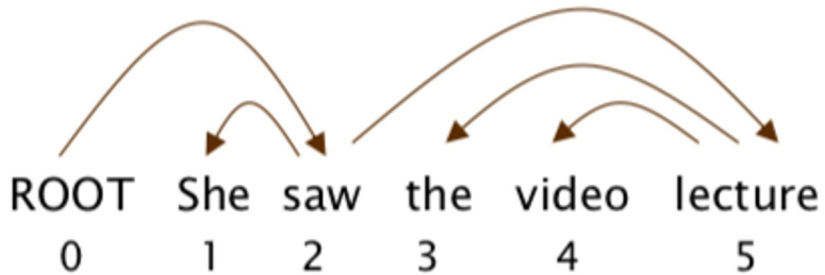
MaltParser

[Nivre and Hall 2005]

- We have left to explain how we choose the next action
 - Answer: Stand back, I know machine learning!
- Each action is predicted by a discriminative classifier (e.g., softmax classifier) over each legal move
 - Max of 3 untyped choices; max of $|R|^2 + 1$ when typed (R: # arc actions)
 - Features: top of stack word, POS; first in buffer word, POS; etc.
- There is NO search (in the simplest form)
 - But you can profitably do a beam search if you wish (slower but better): You keep k good parse prefixes at each time step
- The model's accuracy is *fractionally* below the state of the art in dependency parsing, but
- It provides very **fast linear time parsing**, with great performance

Evaluation of dependency parsing

(labeled) dependency accuracy



$$\text{Acc} = \frac{\text{\# correct deps}}{\text{\# of deps}}$$

$$\text{UAS} = 4 / 5 = 80\%$$

$$\text{LAS} = 2 / 5 = 40\%$$

Gold

1	2	She	nsubj
2	0	saw	root
3	5	the	det
4	5	video	nn
5	2	lecture	obj

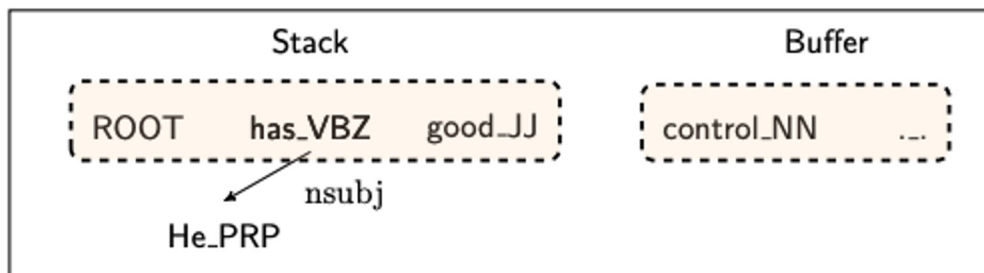
Parsed

1	2	She	nsubj
2	0	saw	root
3	4	the	nn
4	5	video	nsubj
5	2	lecture	ccomp

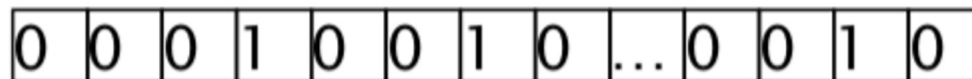
UAS: unlabeled attachment score

LAS: labeled attachment score

Conventional feature representation



binary, sparse
dim = $10^6 \sim 10^7$



Feature templates: usually a combination of 1 ~ 3 elements from the configuration.

Indicator features

$s1.w = \text{good} \wedge s1.t = \text{JJ}$
 $s2.w = \text{has} \wedge s2.t = \text{VBZ} \wedge s1.w = \text{good}$
 $lc(s2).t = \text{PRP} \wedge s2.t = \text{VBZ} \wedge s1.t = \text{JJ}$
 $lc(s2).w = \text{He} \wedge lc(s2).l = \text{nsubj} \wedge s2.w = \text{has}$

Motivation of a neural dependency parser [Chen and Manning 2014]

- Problem 1: sparse
- Problem 2: incomplete representation
- Problem 3: expensive computation
- More than 95% of parsing time is consumed by feature computation

Motivation of a neural dependency parser [Chen and Manning 2014]

- Proposed approach:
 - learn a dense and compact feature representation
- English parsing to Stanford dependencies:
 - UAS = head
 - LAS = head and label

Parser	UAS	LAS	sent. / s
MaltParser	89.8	87.2	469
MSTParser	91.4	88.1	10
TurboParser	92.3	89.6	8
C & M 2014	92.0	89.7	654

[MSTParser](#): a non-projective dependency parser that searches for maximum spanning trees over directed graphs.

[TurboParser](#): a dependency parser with linear programming.

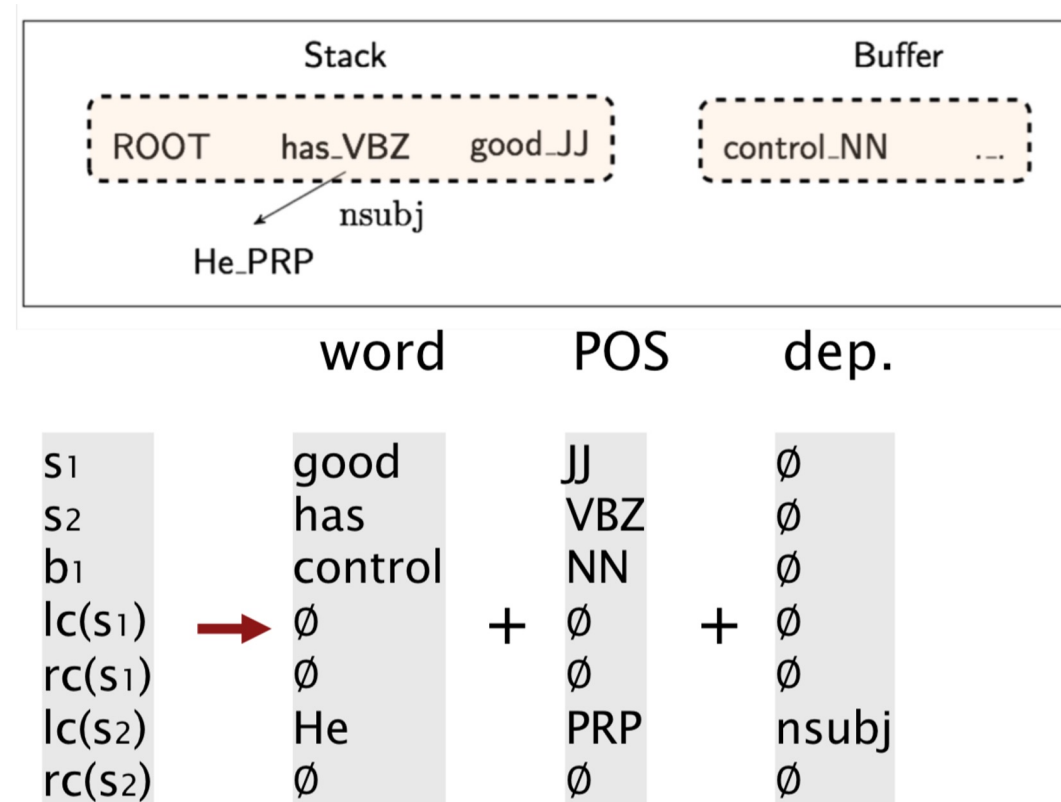
Distributed representations

- Represent each word as a d-dimensional dense vector (i.e., word embedding)
 - Similar words are expected to have close vectors.
- Meanwhile, **part-of-speech tags** (POS) and **dependency labels** are also represented as d-dimensional vectors.
 - The smaller discrete sets also exhibit many semantical similarities.

- ☐ NNS (plural noun) should be close to NN (singular noun)
- ☐ num (numerical modifier) should be close to amod (adjective modifier)

Extracting Tokens and then vector representations from configuration

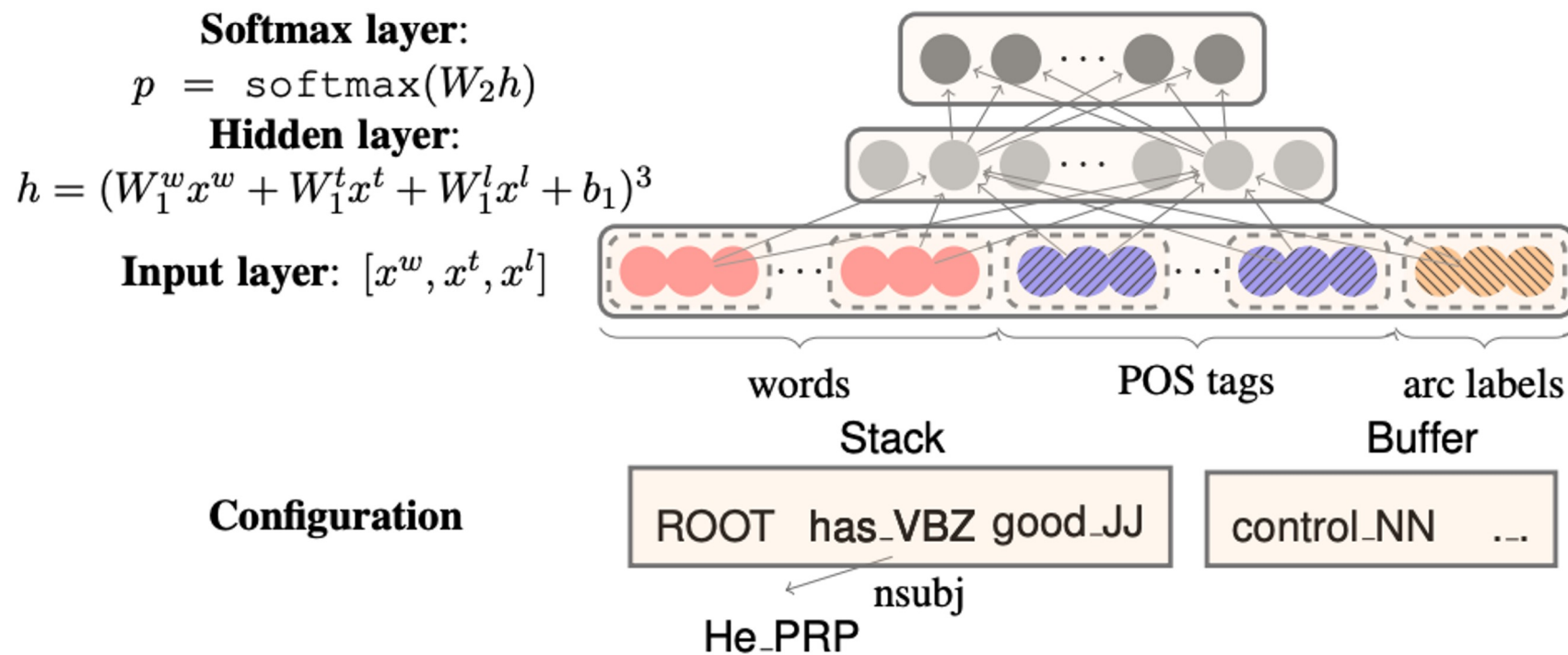
- Extract a set of tokens based on the stack/buffer positions



- We convert them to vector embeddings and concatenate them to get the neural representation of a configuration.

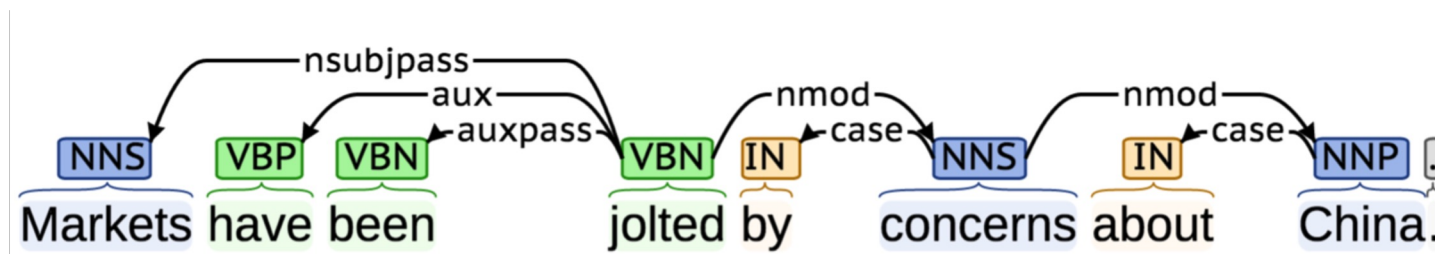
Model architecture

A simple feed-forward neural network multi-class classifier



Dependency parsing for sentence structure

Neural networks can accurately determine **the structure of sentences**, supporting interpretation.



- ❑ [Chen and Manning 2014] was the first simple, successful neural dependency parser.
- ❑ The dense representations (and non-linear classifier) let it outperform other greedy parsers in both accuracy and speed

Further developments in transition-based neural dependency parsing

This work was **further developed and improved** by others

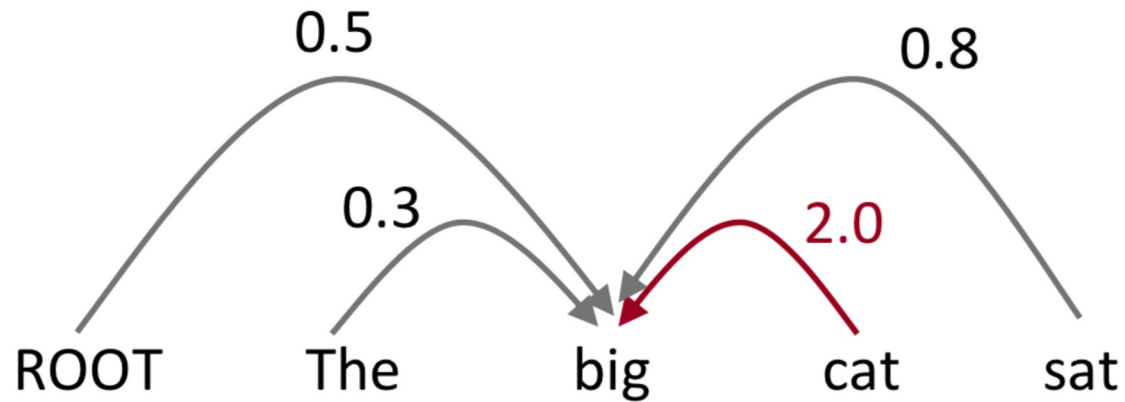
- ❑ Bigger, deeper networks with better tuned hyperparameters
- ❑ Beam search
- ❑ Global, conditional random field (CRF)-style inference over the decision sequence

Method	UAS	LAS (PTB WSJ SD 3.3)
Chen & Manning 2014	92.0	89.7
Weiss et al. 2015	93.99	92.05
Andor et al. 2016	94.61	92.79

Dependency parsing progress: http://nlpprogress.com/english/dependency_parsing.html
<https://ai.googleblog.com/2016/05/announcing-syntaxnet-worlds-most.html>

Graph-based dependency parsers

- Compute a score for **every possible** dependency for each edge
 - Then add an edge from each word to its highest-scoring candidate head
 - And repeat the same process for each other word



e.g., picking the head for “big”

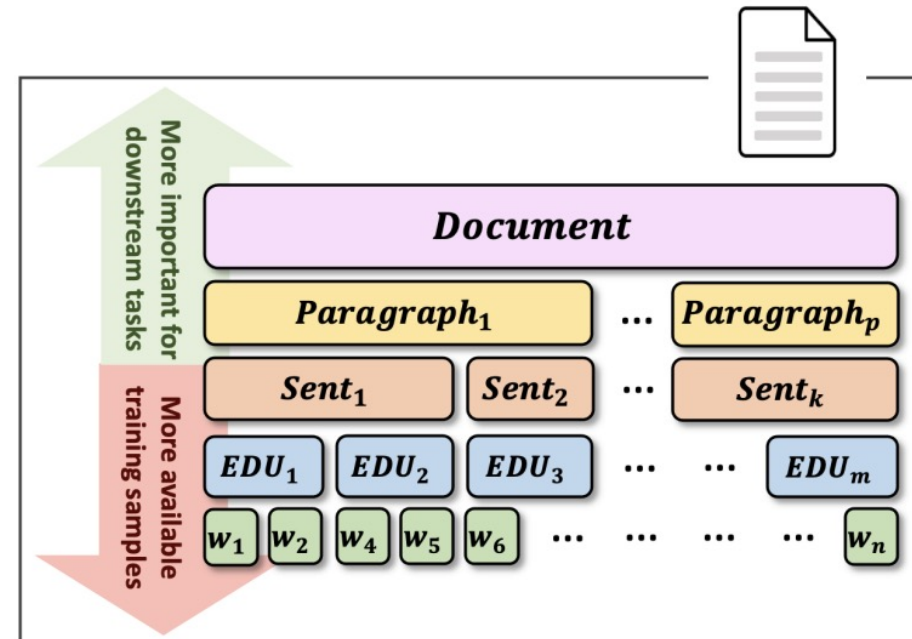
A neural graph-based dependency parser

[Dozat and Manning 2017; Dozat, Qi, and Manning 2017]

- Revived graph-based dependency parsing in a neural world
 - Design a biaffine scoring model for neural dependency parsing
 - Also using a neural sequence model
- Really great results!
 - But slower than simple neural transition-based parsers
 - There are n^2 possible dependencies in a sentence of length n

Method	UAS	LAS (PTB WSJ SD 3.3)
Chen & Manning 2014	92.0	89.7
Weiss et al. 2015	93.99	92.05
Andor et al. 2016	94.61	92.79
Dozat & Manning 2017	95.74	94.08

Discourse



Discourse structures

Discourse is about the structure between sentences.

It's what makes a text more than just a random collection of grammatical sentences.

Consider these two texts:

Coherent: "I am hungry. I am going to the kitchen to get a snack."

The second sentence explains the consequence of the first.

Incoherent: "I am hungry. My car is blue. Trees have leaves."

All sentences are grammatically correct, but they don't form a meaningful whole.

Rhetorical Structure Theory

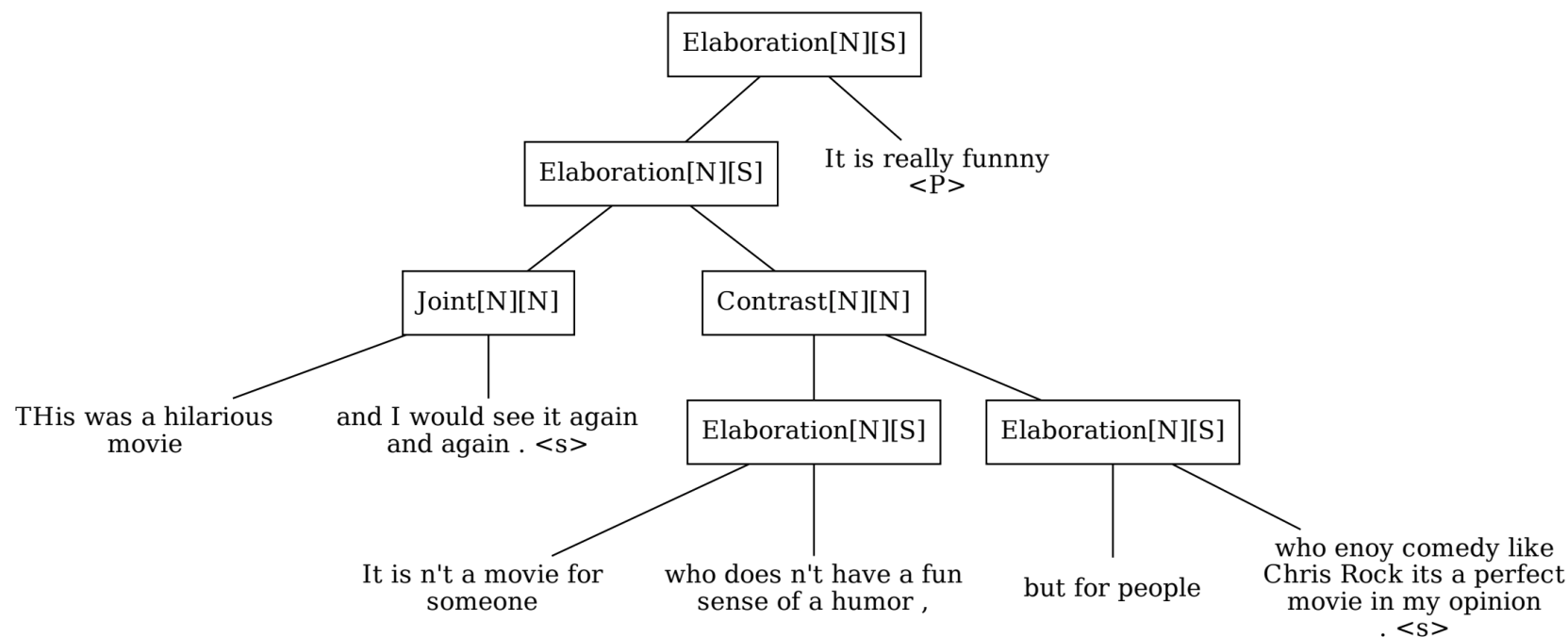
- Core Idea: Texts are not just strings of clauses, but are organized hierarchically. Parts of the text are linked by meaningful relationships.
- The Basic Building Block: A relation between two spans of text:
 - Nucleus (N): The central, most important part of the relation.
 - Satellite (S): Provides supporting, secondary information about the nucleus.
- A text is a tree of these nucleus-satellite relationships.

Examples of rhetorical relations

Relation	Description	Example (Satellite in italics)
Elaboration	Provides more detail about the nucleus.	"I love to visit Paris. It has the best museums."
Evidence	Provides evidence to support the nucleus claim.	"We should leave now. The weather report says it will rain."
Cause	The satellite provides a reason for the nucleus.	"The street is wet because it rained last night."

An RST Tree

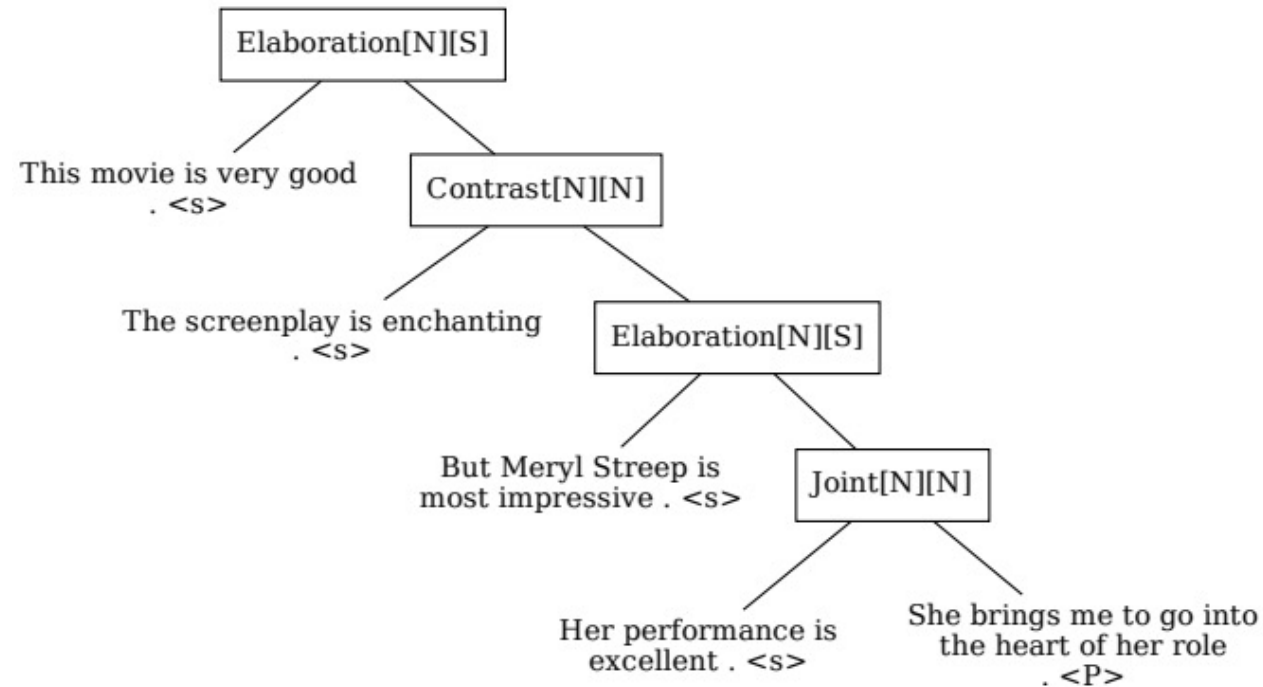
“THis was a hilarious movie, and I would see it again and again. It isn’t a movie for someone who doesn’t have a fun sense of humor, but for people who enoy comedy like Chris Rock it’s a perfect movie in my opinion. It’s really funny.”



An example of Rhetorical Structure Theory parsing from an IMDB movie review. Figure from: Examining the rhetorical capacities of neural language models (2020) Zining Zhu, Chuer Pan, Mohamed Abdalla, Frank Rudzicz., *BlackboxNLP*

An RST Tree

“This movie is very good. The screenplay is enchanting. But Meryl Streep is most impressive . Her performance is excellent . She brings me to go into the heart of her role.”



An example of Rhetorical Structure Theory parsing from an IMDB movie review. Figure from: Examining the rhetorical capacities of neural language models (2020) Zining Zhu, Chuer Pan, Mohamed Abdalla, Frank Rudzicz., *BlackboxNLP*

Discourse structures in news

Summary statement

Effect

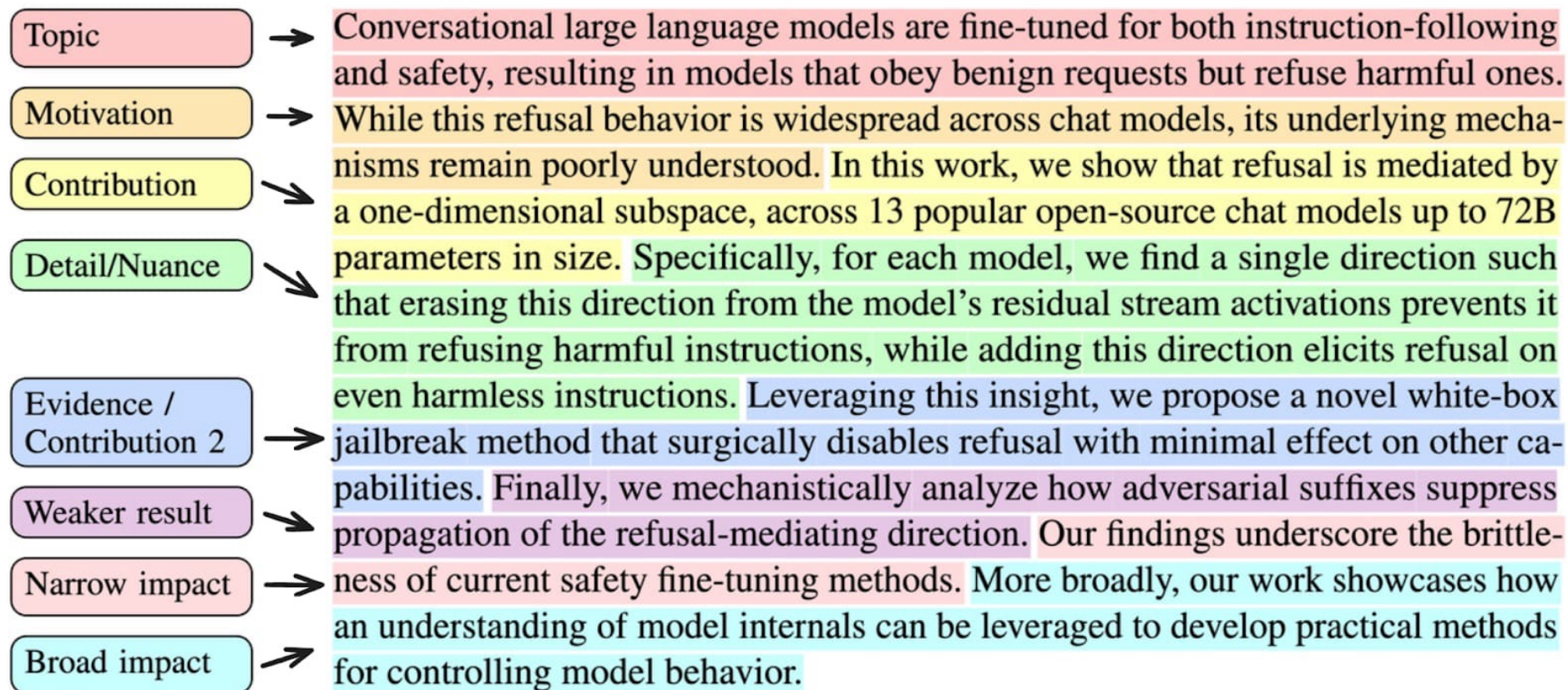
Supporting data

Interpretation

U.S. factory output rose 0.6% in July, easing concerns about an impending recession as automakers ramped up production. The Commerce Department's report showed that machinery and primary metals posted strong gains—up 1.2% and 0.9%, respectively—while there were moderate increases across sectors tied to consumer goods. Economists say the data suggests that industrial demand remains resilient, although supply-chain constraints continue to limit capacity in key segments.

A GPT-generated paragraph with synthetic numbers.

Discourse structures in a paper abstract



From “[Highly Opinionated Advice on How to Write Research Papers](#)”, by Neel Nanda

Discourse parsing algorithms

1. Segment raw text into basic clause-like units: elementary discourse units (EDUs)

An EDU could be a sentence, but usually smaller than that.

2. Build up discourse parsers similarly to syntax parsers.

For example, shift-reduce parsers, neural network based parsers.

Summary

- Syntax
 - Part-of-speech tagging
 - Constituency and dependency parsing
- Discourse
 - Rhetorical structure theory

Course Outcomes

1. Implement gradient descent (GD) and stochastic gradient descent (SGD) techniques for learning problems and understand the theory behind them.
2. Apply word2vec models in real-world text corpora.
3. Understand the neural networks models and backpropagation optimization. Implement neural network models in TensorFlow or others.
4. Understand dependency parsing, recurrent neural networks and their application in NLP.
5. Understand convolutional neural networks and their application in NLP.
6. Understand sequence to sequence models and attention in NLP deep neural networks.
7. Understand the recent advances in NLP, such as transformers, pre-training, fine-tuning, prompting, etc.

Acknowledgements

Slides adapted from Dr. Christopher Manning's Natural Language Processing with Deep Learning at Stanford.

Slides adapted from Dr. Zining Zhu and Dr. Ping Wang's CS 584 slides at Stevens.